Git Practical: Complete Step-by-Step Guide

Git Bash on Windows • Commands, purpose, expected results, and common fixes

What this guide covers

- Create a local repository and commit files
- Modify files, compare changes, and view history
- Create branches, switch branches, and merge
- Create and resolve a merge conflict
- Connect to GitHub, push, clone, and pull

Before starting

These steps use the folder git_practice and branch master, matching your practice. If your branch is named main, use main instead of master in the relevant commands.

If Git asks for your identity, configure it once with your own details:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

The LF/CRLF warning sometimes shown on Windows is a line-ending warning. It does not normally mean the command failed.

Part 1 — Local repository, files, and commits

1. Create and enter the folder

Purpose: Create the working directory and move into it.

```
mkdir git_practice
cd git_practice
```

Note: If the folder already exists, do not create it again. From your home directory, use cd git_practice.

2. Initialize Git

Purpose: Start a local Git repository.

```
git init
```

Note: The prompt may show (master), which is the current branch.

3. Create a file

Purpose: Create notes.txt with the first line and display it.

```
echo "Hello World" > notes.txt
cat notes.txt
```

Note: > creates or overwrites a file; cat displays its contents.

4. Check status

Purpose: See whether Git is tracking the file.

```
git status
```

Note: Before staging, notes.txt should appear as untracked.

5. Stage and commit

Purpose: Stage the file and save the first snapshot.

```
git add notes.txt
git commit -m "First Commit"
```

Note: git add stages the file; git commit saves staged changes with a message.

6. Modify the file

Purpose: Append a second line without deleting the first.

```
echo "Learning Git" >> notes.txt
```

Note: >> appends text to the file.

7. Inspect unstaged changes

Purpose: Compare the working file with the last commit.

```
git diff
```

Note: A + line is added content. This command does not commit changes.

8. Stage and commit the modification

Purpose: Review staged changes, then save them.

```
git add notes.txt
git diff --staged
git commit -m "Updated Notes"
```

Note: git diff --staged shows changes already staged for commit.

9. View history and status

Purpose: Review commits and check that there are no pending changes.

```
git log
git log --oneline
git status
```

Note: git log is detailed; --oneline is compact. A clean result says nothing to commit, working tree clean.

Part 2 — Branching and normal merging

10. Create and list a branch

Purpose: Create testing and list local branches.

```
git branch testing
git branch
```

Note: The asterisk (*) marks the current branch.

11. Switch and commit on testing

Purpose: Make a separate change on testing.

```
git switch testing
echo "Testing branch changes" >> notes.txt
git add notes.txt
git commit -m "Added testing changes"
```

Note: Commit the change while you are on testing.

12. Switch back and merge

Purpose: Bring testing's commit into master.

```
git switch master
git merge testing
```

Note: Run the merge while on the destination branch. Fast-forward means the normal merge succeeded.

13. Verify

Purpose: Check history and status.

```
git log --oneline
git status
```

Note: After a successful merge and no uncommitted changes, status should be clean.

Part 3 — Merge conflict resolution

A conflict can occur when two branches change the same part of a file differently. The steps below deliberately create one.

14. Create the original file on master

Purpose: Create and commit the starting version.

```
git switch master
echo "Original line" > conflict.txt
git add conflict.txt
git commit -m "Added conflict file"
```

15. Change it on feature

Purpose: Create a branch and commit one version.

```
git switch -c feature
echo "Feature change" > conflict.txt
git add conflict.txt
git commit -m "Changed file on feature"
```

16. Change the same line on master

Purpose: Commit a different version on master.

```
git switch master
echo "Master change" > conflict.txt
git add conflict.txt
git commit -m "Changed file on master"
```

17. Trigger the conflict

Purpose: Attempt to merge feature into master.

```
git merge feature
```

Note: Git should show CONFLICT (content); the prompt may show (master|MERGING).

18. Edit the conflicted file

Purpose: Choose the final content and remove conflict markers.

```
notepad conflict.txt
```

Note: Replace the whole file with the following line. Save with Ctrl+S and close Notepad:

```
Final resolved line
```

19. Complete the resolution

Purpose: Stage the fixed file, commit, and verify.

```
git add conflict.txt
git commit -m "Resolved merge conflict"
git status
```

Note: The prompt should no longer show MERGING. The working tree should be clean.

Important: remove <<<<<<<, =======, and >>>>>>> markers from the file before staging it. Run Git commands in Git Bash after saving and closing Notepad, not inside Notepad.

Part 4 — GitHub remote and push

These steps need internet access.

20. Create a GitHub repository

Purpose: On GitHub, sign in, choose New repository, and create an empty repository named git_practice.

```
Repository URL format:
https://github.com/YOUR_USERNAME/git_practice.git
```

Note: Because your local project already exists, leave README, license, and .gitignore unchecked for this practice.

21. Add the remote

Purpose: Connect your local repository to GitHub and verify the URL.

```
git remote add origin https://github.com/YOUR_USERNAME/git_practice.git
git remote -v
```

Note: Replace YOUR_USERNAME with your GitHub username. origin is the common remote name.

22. Push commits

Purpose: Upload the local master branch and set upstream tracking.

```
git push -u origin master
```

Note: If the branch is main, use git push -u origin main. Internet and GitHub authentication are required.

Your actual repository URL from the practice:

```
https://github.com/AbhishekJadhav111/git_practice.git
```

Part 5 — Clone and pull

23. Clone into a new folder

Purpose: Download the repository and its commit history.

```
cd ..
git clone https://github.com/AbhishekJadhav111/git_practice.git git_practice_copy
```

Note: Use a new destination folder if git_practice already exists. Cloning requires internet.

24. Enter the cloned folder and verify

Purpose: Inspect files, status, history, and remote.

```
cd git_practice_copy
ls
git status
git log --oneline
git remote -v
```

Note: Run cd git_practice_copy from the parent directory (such as ~), not while inside ~/git_practice.

25. Pull updates

Purpose: Fetch and integrate changes from the remote branch.

```
git pull origin master
```

Note: Use main if that is the remote branch. 'Already up to date' is a successful result meaning there were no new remote changes to integrate. Pull requires internet.

Command reference

Command	Purpose
`mkdir foldername`	Create a directory
`cd foldername`	Enter a directory
`git init`	Initialize a local repository
`echo "text" > file.txt`	Create/overwrite a file
`echo "text" >> file.txt`	Append text to a file
`cat file.txt`	Display file contents
`git status`	Show repository status
`git add file.txt`	Stage changes
`git commit -m "message"`	Save staged changes as a commit
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git log`	Show detailed commit history
`git log --oneline`	Show compact history
`git branch`	List local branches
`git branch testing`	Create testing branch
`git switch testing`	Switch to existing branch
`git switch -c feature`	Create and switch to a new branch
`git merge feature`	Merge feature into current branch
`git remote -v`	Show remote URLs
`git push -u origin master`	Upload commits and set upstream
`git clone URL`	Create a local copy of a remote repository
`git pull origin master`	Fetch and integrate remote changes
`git fetch`	Download remote updates without integrating them into the current branch

Common issues and fixes

- mkdir git_exam showed File exists because that folder was already present. Use another name or enter the existing folder.
- git clone failed when git_practice already existed and was not empty. Cloning into git_practice_copy fixed it.
- After cd .. from ~/git_practice, you were back in your home directory. Run cd git_practice_copy there; do not try it while inside ~/git_practice.

- git pull origin master returned Already up to date. This is not an error; there were no newer remote commits.
- LF/CRLF warnings are common on Windows and do not by themselves indicate failure.

Offline exam checklist

- Works offline: init, add, commit, diff, log, branch, switch, merge, and merge-conflict resolution.
- Needs internet: creating a repository on GitHub, pushing, cloning from GitHub, and pulling from GitHub.
- Set your Git name and email before the exam if needed.
- After resolving a conflict, save and close the file before running git add and git commit.
- Finish with git status and verify that the working tree is clean.